Original question:
// ---------- //
Data synchronisation tool
You are tasked with designing a utility that monitors a specific set of directories on your machine, and synchronises the contents to a remote collection server. However, you have limited bandwidth to communicate with the server, so the utility must be highly efficient.

You can assume that you will have a corresponding server-side application that you can also control, but this question is primarily concerned with the client-side of the application and associated considerations.

Core requirements:

Change Detection: Only files that have been created or modified since the last successful sync should be transferred.
Bandwidth Management: The utility must minimise the amount of data sent over the wire.
Reliability: The system must handle unstable network connections. If a transfer is interrupted, it should not require restarting the entire process.
Integrity: The server must be able to verify that the file received is an exact copy of the file on your client machine.

Submission requirements:

Design Document: Provide a 1-2 page PDF or Word document outlining your architectural approach.
Code (optional): Provide a small code snippet demonstrating your approach to one of the core requirements, and why you chose this component. This could include aspects such as:
Detecting how a file has changed.
How to manage the transfer of a single file.

If you choose to do the optional coding component, it is recommended that you upload to GitHub and provide a link for the assessors to view.
// ---------- //

Assumptions:
"Monitors a specific set of directories"
Assuming this means we are watching a directory, and all of it's contents for file / directory changes. "specific set" implies that we can input multiple locations to sync up

"synchronises the contents to a remote collection server"
This implies that we are mirroring the directories in a location we choose.

Limited bandwidth:
We have to minimize the amount of changes we send across the network, so we will prioritize compute and using algorithms to minimize data being sent.

"You can assume that you will have a corresponding server-side application that you can also control, but this question is primarily concerned with the client-side of the application and associated considerations."

This implies that the tool lives on our local host, and sends the data to the remote location in question. Assumming normal usage is controlling the tool remotely through the server side, however the actual functionality is done by the client side. The developers of the question want us to focus on the client side considerations rather than the server side.

Core requirements:

Change Detection: Only files that have been created or modified since the last successful sync should be transferred.
Bandwidth Management: The utility must minimise the amount of data sent over the wire.
Reliability: The system must handle unstable network connections. If a transfer is interrupted, it should not require restarting the entire process.
Integrity: The server must be able to verify that the file received is an exact copy of the file on your client machine.

The core requirements seem relatively straight forward

// ---------- //
Feedback from qwen 3.8 and Claude:

"mirrored vs collection":
One thing a bit confusing is the definition of a "collection server" and how the tool "synchronises the contents to a remote collection server". A gap in this is do we mirror or do we accumulate.

The designer of the question states in the core requirements:
"Change Detection: Only files that have been created or modified since the last successful sync should be transferred."

This implies deletions should not be tracked and the tool accumulates files at the collection server. It is worth considering if some sort of "tombstone" marking that outlines if a file we have collected is deleted, when that file was deleted.

It also lays out another boundary: What is the maximum accumulation size?

"No concrete bandwidth mechanism":

This is to be expected as I did not explain exactly how we would limit bandwidth.
There are several considerations of how we can limit bandwidth.
1. rsyncs rolling checksum block diff algorithm.
2. aria2 resumable chunked downloader.

Answer the question "What do we actually put on the wire when file X changes 200 bytes out of 2gb?"

" 'modified' is undefined"
This is an interesting one. using touch on a file might "modify" it, but not actually change the contents. There seem to be several options pointed to by the AI
mtime vs mtime + size vs content hash.

"Controlling the tool remotely through the server side"
sounds awkward and I agree.

Round 3:

There is some more ambiguity on deletions that is worth thinking about. Firstly, it is important to note that the traditional "Tombstone" is a concept from the CRDT paradigm which deals with concurrency. We are assuming concurrency is not an issue, and we have just one client per point on a collection server.

It also begs the question on version control: Technically a file can "still exist" even if we delete all of it's contents. Should some sort of versioning be built into the tool or be left up to the collection server? I am going to assume that this is a server side problem and not to focus on this.

As well as versioning control, maximum accumulation size is a server side policy question. The important thing is for the client to have some sort of awareness to not send anymore data if it has reached the maximum accumulation size, presumably erroring.

question - what meta data do we preserve? i.e., perms, ownership timestamps, symlinks.
I want to say just timestamps ?

checksum for comparison between files to ensure integrity

Assuming we are syncing literally every file in the directory

Encryption; the original design question does not mention encryption

Round 4:
What are we actually getting graded on? We have listed all of the considerations above but do they actually answer any of the design requirements.
We should focus on answering the core requirements foremost, and then discuss boundaries and edge-cases second.

How do we learn that something has changed?
the options are
a.) OS filesystem API (inotify linux, FSEvents mac, ReadDirectoryChangesW on Windows)
b.) Periodic full recursive scan comparing against last known state (cron job potentially)
c.) both

Apparently watchers (the event watchers mentioned above in a.) are cheap, but can lie. For example inotify drops events on queue overflow
Claude suggested an architecture that uses both, where the watcher is a hint to do a scan, whilst the scan runs a schedule to guarantee a scan occurs.

We need some sort of "manifest" which keeps track of what files and directories have been synced so far between the client and server. This is a gap in my knowledge and I am not entirely sure how this will look :/

hash-as-arbiter (arbiter being person with power): If we use the hash as the judgement whether a file is modified, what are the drawbacks with that?
Hashing whole files, vs blocks vs both

- silently missing data changes is the worst possible failure for the tool; this needs to be mentioned

Caching what the server has: The client has to "know" what the server has so it might as well cache the results.

Who owns state sync? and what happens if the client loses it?
At the moment we are assuming a local manifest exists but haven't said where it lives, what's in it, what happens if it is deleted or corrupted, or the process is killed mid sync

Transport and resume primitive:
it is stated "If a transfer is interrupted, it should not require restarting the entire process."
we have several options to address this:
1. HTTP with byte range resume per file
2. gRPC bidirectional streaming
3. Custom protocol over raw TCP

I am going to go with a custom protocol over raw TCP
Need to mention TLS somehow, and encryption.

Integrity, and the file changing while you are reading it:
requirement states "The server must be able to verify that the file received is an exact copy of the file on your client machine."
There is a race condition where a file is modified while you are chunking and uploading it. a potential solution is to read the file, verify the hash of the read file and the file on disk again, then mark it as dirty if it doesn't match up.

Potential solution: per-chunk hash verified by the server on receipt.
BLAKE3. Also potential to store an algorithm in the manifest to allow for easy migration to another algorithm.

Another edge case: file renames. a potential blunder is reuploading the whole file if it has been renamed

So lets say collection is append-only, we report deletions as meta data associated with the file in the manifest (also meaning we can skip of synchronizing them, although I just thought of an edge case where a file is deleted, and then a new file is created with that same name. Going to assume this is a server side problem of managing deletions).
Renames will just be updating the manifest.
Regarding the edge case:
apparently a new file at path P has a different hash to a tombstoned entry so it syncs as new content? need to verify.